



Sistema de gestión operativa para panificadora artesanal
Desarrollado por René Magaña Vega

¿Qué es OvenOps?

OvenOps es un sistema de gestión diseñado para cubrir las operaciones diarias de una panificadora artesanal en Tangancicuaro, Michoacán. Nació de una necesidad real: llevar un control confiable de ventas, producción, inventario y pagos sin depender de hojas de cálculo o registros a mano.

El sistema corre desde terminal y persiste toda la información en archivos CSV, lo que lo hace portable, ligero y sin dependencias externas más allá de Python.

Funcionalidades actuales

- **Registro de ventas** — por pieza y por caja, con cálculo automático de cambio
- **Cierre de caja** — resumen del turno con total acumulado y conteo de unidades
- **Recuperación ante fallos** — si el programa se cierra inesperadamente, las ventas no contabilizadas se recuperan automáticamente al reiniciar
- **Control de inventario** — snapshot diario que registra producción, merma y ventas para calcular el inventario real al cierre
- **Registro de producción** — cantidad de piezas y cajas producidas por día con estado de pago
- **Pago semanal de producción** — calcula automáticamente el pago de la semana anterior descontando merma
- **Histórico de cortes** — consulta de todos los cierres contables anteriores

Estructura del proyecto

```
OvenOps/
|
|-- Main.py      # Punto de entrada, menú principal y flujo de la aplicación
|-- ventas.py    # Lógica de ventas: registro, cálculo de precio y cambio
|-- datos.py     # Persistencia: lectura y escritura de CSV
|-- production.py # Gestión de producción y cálculo de pago semanal
|-- inventario.py # Control de inventario con snapshots diarios
|-- utils.py     # Funciones puras de cálculo compartidas
|-- config.py    # Precios centralizados (única fuente de verdad)
|
|-- test_panificadora.py # Suite de tests con pytest (21 casos)
|
|-- ventas_individuales.csv # Generado automáticamente
|-- cortes.csv             # Generado automáticamente
|-- production.csv         # Generado automáticamente
|-- inventario.csv         # Generado automáticamente
|-- pagos.csv              # Generado automáticamente
```

Instalación y uso

Requisitos

- Python 3.10 o superior
- Windows 10/11

Pasos

1. Clona o descarga el repositorio

```
bash
git clone https://github.com/tu-usuario/ovenops.git
cd ovenops
```

2. No se requieren dependencias externas para correr el sistema. Los CSV se generan automáticamente en el primer uso.
3. Ejecuta el programa

```
bash
python Main.py
```

Para correr los tests

```
bash
python -m pip install pytest
python -m pytest test_panificadora.py -v
```

Configuración de precios

Todos los precios están centralizados en ([config.py](#)). Si los precios cambian, solo hay que modificar ese archivo:

```
python
precio_pieza      = 11 # Precio unitario al público
precio_pieza_reparo = 8  # Precio para reparadores
precio_caja_publico = 140 # Caja al público
precio_caja_reparo = 130  # Caja para reparadores
precio_pieza_produccion = 2.5 # Pago al productor por pieza
precio_caja_produccion = 25  # Pago al productor por caja
```

Decisiones técnicas

¿Por qué CSV y no una base de datos?

El sistema está pensado para correr en una computadora sin configuración adicional. SQLite habría sido la siguiente opción, pero requería conocimiento técnico del operador para hacer respaldos o consultas directas. Los CSV son legibles, editables en Excel si se necesita una corrección manual, y no requieren servidor.

¿Por qué la separación en módulos?

Desde el inicio se pensó en una transición futura a una interfaz gráfica (tkinter). Por esa razón toda la lógica de negocio vive separada de la presentación en ([Main.py](#)); cambiar la interfaz no requiere tocar las funciones de cálculo ni de persistencia.

Sistema de recuperación anti-fallos

Cada venta se guarda en CSV con estado ([abierto](#)) en el momento de registrarse. Al ejecutar el cierre de caja, las ventas pasan a ([cerrado](#)). Si el programa se interrumpe antes del cierre, al reiniciar detecta las ventas ([abiertas](#)) y las recupera automáticamente a la sesión activa.

Roadmap — mejoras plancadas

- ☐ **Interfaz gráfica con tkinter** — la arquitectura actual ya está preparada para esta transición, toda la lógica está desacoplada de la presentación
- ☐ **Módulo de reparto** — registro del pan que se lleva el repartidor, liquidación al día siguiente con ventas, sobrante y merma, integrado al sistema de pagos y al inventario
- ☐ **Exportación de reportes** — generación de tablas y resúmenes para análisis externo
- ☐ **Análisis de datos y forecasting** — una vez recopilados suficientes datos históricos, se planea entrenar modelos de series de tiempo para predicción de demanda, aprovechando los CSV como fuente de datos estructurada

Tests

El proyecto incluye una suite de 21 tests unitarios que cubren las funciones de lógica de negocio más críticas:

Módulo	Casos cubiertos
utils.py	Resumen de ventas piezas, cajas, mixto y lista vacía
inventario.py	Producción, ventas, merma, inventario en cero, inventario negativo
ventas.py	Cálculo de precio, cambio, pago exacto y cero unidades
production.py	Pago sin merma, con merma parcial y semana completa

Los tests de funciones que interactúan con el sistema de archivos (lectura/escritura de CSV) están fuera del alcance de esta suite por diseño, ya que requieren técnicas de mocking que se implementarán en una versión futura.

Autor

René Magaña Vega
Estudiante de ingeniería — área ML / Data Science
Proyecto personal desarrollado entre enero y febrero de 2026

Este proyecto fue construido para resolver un problema real. La panificadora existe, el operador existe, y los datos que genera este sistema serán la base para futuros experimentos de análisis y predicción de demanda.